



TECH CHALLENGE — FASE 3

Ecosistema de Microserviços Hospitalares

Documentação técnica da AgendamentoAPI, da HistoricoAPI e da NotificacaoAPI. Reúne o agendamento de consultas com autenticação JWT e mensageria, a consulta do histórico do paciente por GraphQL e a notificação dos pacientes por e-mail, sobre um PostgreSQL e um RabbitMQ compartilhados.

DISCIPLINA

Arquitetura e Desenvolvimento em Java

TURMA E GRUPO

Turma 12ADJT · Grupo 46

INTEGRANTES

Gustavo Chauar Correa (RM 370978)
Eduardo Mendes Germano (RM 371760)
Davi Faustino Souza Lima (RM 373019)
Caroline Da Silva Brunheti (RM 372744)
Roberto Domingos Locatelli (RM 370928)

REPOSITÓRIOS

github.com/JavaPosTech/AgendamentoAPI
github.com/JavaPosTech/HistoricoAPI
github.com/JavaPosTech/NotificacaoAPI

ENTREGA E VERSÃO

Setembro de 2026 · versão 1.0.0

Sumário

1. Introdução	3
Descrição do problema · Objetivo · Os três microsserviços	3
2. Arquitetura do Sistema	4
Camadas · Tecnologias · Decisões · Topologia de contêineres	4
3. AgendamentoAPI: Serviço de Agendamento	7
Domínios · Estrutura de pastas · Segurança · Endpoints · Regras de negócio · Mensageria · Exemplos	7
4. HistoricoAPI: Serviço de Histórico	12
Estrutura de pastas · GraphQL · Endpoints · Erros previstos · Exemplos	12
5. NotificacaoAPI: Serviço de Notificação	14
Estrutura de pastas · Fluxo · Contrato de eventos · Resiliência · Lembretes · Banco de dados · Exemplo	14
6. Configuração e Execução	18
Docker Compose · Ordem de subida · Passo a passo · URLs e portas	18
7. Qualidade do Código	22
Boas práticas · SOLID e DRY · Testes e cobertura · Integração contínua	22
8. Repositórios do Código	23

1. Introdução

Descrição do problema

Em um ambiente hospitalar é essencial contar com sistemas que garantam o **agendamento eficaz de consultas**, o **gerenciamento do histórico do paciente** e a **comunicação assíncrona** entre os serviços que reagem a esses eventos. O sistema deve ser acessível a diferentes tipos de usuários (administradores, médicos, enfermeiros, recepcionistas e pacientes), com acesso controlado e funcionalidades específicas para cada perfil.

Para atender a esse cenário, a solução da fase é decomposta em **microsserviços**. O atendimento gera informação em pontos distintos do fluxo: o cadastro das pessoas, o agendamento das consultas, o registro do histórico clínico e o aviso ao paciente. Cada recorte é responsabilidade de um serviço próprio, com o seu ciclo de build, imagem e implantação independentes, apoiados na mesma infraestrutura: um servidor **PostgreSQL** e um broker **RabbitMQ** compartilhados.

Objetivo do projeto

Desenvolver um backend **modular e seguro**, aplicando boas práticas de autenticação, autorização, comunicação entre serviços e containerização, de modo que cada serviço possa escalar e ser substituído de forma independente. Concretamente, a fase entrega três microsserviços complementares:

- **AgendamentoAPI**: API REST que cadastra pessoas (médicos, enfermeiros, recepcionistas e pacientes), autentica usuários por JWT, agenda e remarca consultas, registra o histórico clínico e publica eventos de agendamento em uma fila RabbitMQ. É a **dona do schema** do banco compartilhado.
- **HistoricoAPI**: API GraphQL, somente leitura, que consolida em uma única consulta os dados cadastrais do paciente, o histórico clínico e as consultas agendadas, lendo o mesmo banco sem duplicar a responsabilidade de evolui-lo.
- **NotificacaoAPI**: serviço sem interface HTTP que **consome os eventos de agendamento** da fila, envia e-mail ao paciente quando a consulta é marcada ou remarcada e dispara um **lembrete na véspera**. Guarda o histórico de envios na própria tabela, em um banco dedicado.

Divisão de responsabilidades. A AgendamentoAPI concentra toda a escrita do domínio hospitalar e é a dona do schema compartilhado, criando as tabelas e a carga inicial. A HistoricoAPI apenas mapeia esse schema e responde por consultas. A NotificacaoAPI não lê as tabelas da AgendamentoAPI: tudo o que precisa chega no evento, e o que grava fica na tabela **notificacao**, em um banco próprio com migrations próprias. Essa separação evita que dois serviços disputem a evolução do mesmo banco.

Os três microsserviços em um olhar

Serviço	Responsabilidade	Interface	Porta (host)
AgendamentoAPI	Cadastros, autenticação JWT, agendamento de consultas, histórico e publicação de eventos. Dona das migrations do schema compartilhado.	REST + OpenAPI	9027
HistoricoAPI	Consulta consolidada do histórico do paciente. Somente leitura.	GraphQL	9028
NotificacaoAPI	Consumo dos eventos de agendamento, e-mail ao paciente e lembrete da véspera. Sem rotas HTTP.	RabbitMQ + SMTP	Sem porta

2. Arquitetura do Sistema

Descrição da arquitetura

Os três serviços seguem a **arquitetura em camadas** do ecossistema Spring, com direção de dependência única: cada camada conhece apenas a imediatamente abaixo. O código é organizado **por camada e, dentro de cada camada, por domínio**, então a estrutura de pacotes revela ao mesmo tempo a responsabilidade técnica e o assunto de negócio. Na AgendamentoAPI, criar um novo domínio significa replicar a fatia vertical completa (entidade → repositório → mapper → request/DTO → serviço → controller → migration → testes). Na NotificacaoAPI não há controller: a entrada acontece pelo listener da fila e por um job agendado, e as demais camadas seguem o mesmo desenho.

Camadas da aplicação

Camada	Responsabilidade
Controller	Fronteira da aplicação. Na AgendamentoAPI, endpoints REST que delegam ao serviço. Na HistoricoAPI, o resolver GraphQL. Na NotificacaoAPI, o listener RabbitMQ e o job agendado. Sem regra de negócio.
Service	Regra de negócio, orquestração, validações, transações (@Transactional) e logging.
Repository	Acesso a dados declarativo, por interfaces do Spring Data JPA.
Model	Entidades JPA, DTOs de saída (records imutáveis), records de request e de evento, e mappers MapStruct.
Exceptions / Config	Exceções de negócio, tratamento centralizado de erros e configuração (segurança, banco, Swagger, RabbitMQ).

Tecnologias utilizadas

Item	Tecnologia	Versão
Linguagem	Java	21
Framework	Spring Boot	4.0.5
Build	Gradle (Kotlin DSL)	9.7
Banco de dados	PostgreSQL	18
Migrações	Flyway, na AgendamentoAPI e na NotificacaoAPI	—
Interface (AgendamentoAPI)	Spring MVC (REST) + SpringDoc OpenAPI	3.0.2
Interface (HistoricoAPI)	Spring for GraphQL + SpringDoc OpenAPI	3.1.0
Segurança	Spring Security + JWT, na AgendamentoAPI	0.12.6
Mensageria	Spring AMQP + RabbitMQ: publicação na AgendamentoAPI, consumo na NotificacaoAPI	4
E-mail	Spring Mail (SMTP com STARTTLS), na NotificacaoAPI	—
Resiliência	Spring Retry, na NotificacaoAPI	2.0.13
Tarefas agendadas	Spring Scheduling (@Scheduled), na NotificacaoAPI	—
Mapeamento	MapStruct, na AgendamentoAPI e na NotificacaoAPI	1.6.3
Persistência	Spring Data JPA / Hibernate	—
Logging	Log4j2 via SLF4J (Logback excluído)	—
Testes e cobertura	JUnit 5, Mockito, JaCoCo	—
Containerização	Docker / Docker Compose	—

Decisões de arquitetura

- **Banco compartilhado com dono único do schema.** As migrations Flyway do schema compartilhado pertencem à AgendamentoAPI, que cria as tabelas e a carga inicial. A HistoricoAPI não possui **ddl-auto** nem Flyway em produção, apenas mapeando o schema existente. Assim há uma fonte única de verdade e nenhum serviço disputa a evolução do banco.
- **Banco dedicado para a NotificacaoAPI.** A NotificacaoAPI usa o mesmo servidor PostgreSQL, mas um banco separado (**notificacao**). Os dois serviços têm migrations **V1.0** e **V1.1** e usam a tabela padrão

flyway_schema_history, então no mesmo banco o Flyway encontraria versões com checksum diferente e interromperia a inicialização.

- **REST para escrita, GraphQL para leitura profunda.** O agendamento é um domínio de escrita com regras e autorização por perfil, natural em REST. O histórico é um agregado profundo (paciente + histórico + consultas). Com GraphQL o cliente declara exatamente os campos desejados, em uma requisição, eliminando *over-fetching* e chamadas encadeadas.
- **Comunicação assíncrona por eventos.** Ao criar ou remarcar uma consulta, a AgendamentoAPI publica um evento em uma exchange RabbitMQ, e a NotificacaoAPI o consome para avisar o paciente. Quem agenda não conhece quem reage, e um novo consumidor entra sem alterar a AgendamentoAPI.
- **Segurança stateless com JWT.** A sessão é **STATELESS**, o token carrega o perfil do usuário, e a autorização é declarada por método HTTP no **SecurityConfig**.
- **Carregamento explícito de relacionamentos.** Com **open-in-view: false** não há sessão JPA aberta na camada web, então as associações necessárias são carregadas dentro da transação, evitando N+1 e **LazyInitializationException**.
- **Containerização reprodutível.** Build multi-stage no Dockerfile (**gradle:jdk21-alpine** compila, **eclipse-temurin:21-jre-alpine** executa) e orquestração por Docker Compose sobre uma rede compartilhada.

Topologia de execução em contêineres

Os serviços da fase compartilham um único PostgreSQL e um único RabbitMQ: a AgendamentoAPI publica os eventos e a NotificacaoAPI os consome. A infraestrutura sobe de forma independente e cria a rede **shared-net**, e cada serviço entra nela como rede externa.

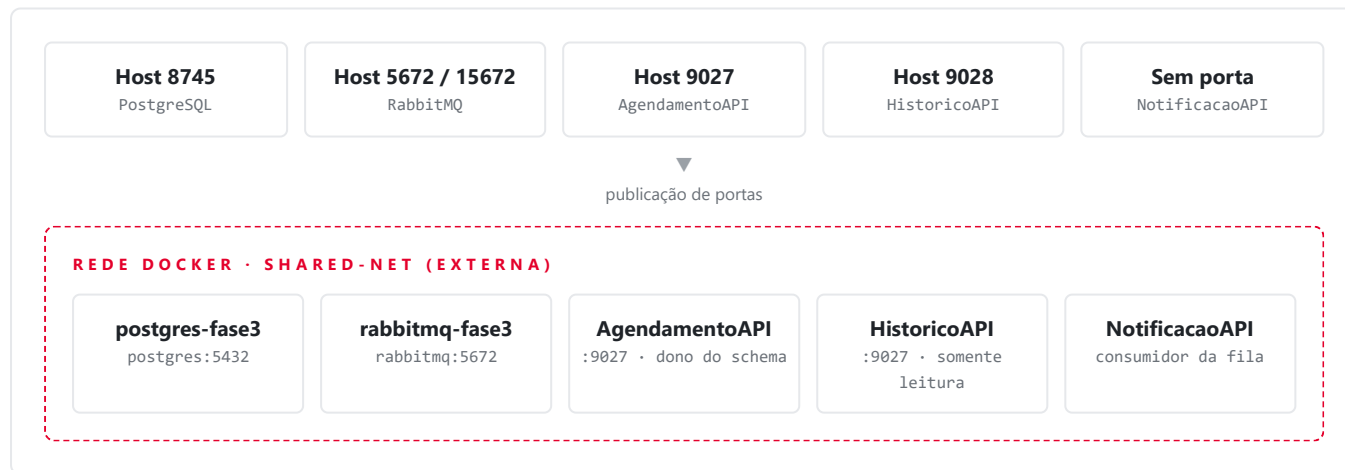


Figura 1. Contêineres e rede compartilhada da Fase 3

Dentro da rede, o banco é sempre alcançado pelo hostname **postgres** na porta interna **5432** e o broker por **rabbitmq:5672**. As portas **8745**, **5672** e **15672** são apenas exposição no host. Como as APIs HTTP escutam em **9027** dentro do próprio contêiner, elas se diferenciam pela porta publicada: a AgendamentoAPI em **9027** e a HistoricoAPI em **9028**. A NotificacaoAPI não publica porta, porque não recebe requisições HTTP.

Ordem de inicialização. A infraestrutura cria a rede **shared-net**, e o Compose de cada serviço a consome como rede externa. Por isso a infraestrutura precisa subir primeiro, e a AgendamentoAPI deve rodar antes da HistoricoAPI contra um banco novo, para que as migrations criem o schema que ela consome. A NotificacaoAPI não depende desse schema: o Compose dela cria o banco **notificacao** antes de iniciar a aplicação, e a fila é declarada com os mesmos argumentos pelos dois serviços, então tanto faz qual deles sobe primeiro.

3. AgendamentoAPI: Serviço de Agendamento

API REST responsável pela escrita do domínio: cadastros, autenticação, agendamento de consultas e histórico clínico. Todas as rotas são relativas ao context path **/AgendamentoAPI**, que diferencia maiúsculas de minúsculas.

Domínios

Cada domínio possui a fatia vertical completa (entidade, repositório, mapper, request/DTO, serviço, controller e testes):

- **usuario / auth:** credenciais e login, com a senha gravada em BCrypt e nunca exposta.
- **medico, enfermeiro, recepcionista:** profissionais, com criação, edição e exclusão restritas ao administrador.
- **paciente:** cadastrado pela recepcionista e excluído pelo administrador.
- **agendamento:** marcação, remarcação e cancelamento de consultas, com regras de agenda.
- **historicopaciente:** registro clínico do paciente, consumido pela HistoricoAPI.
- **tipousuario, situacaocadastro:** tabelas de domínio espelhadas por enums de IDs fixos.

Estrutura de pastas

```
src/main/java/br/com/fiap/agendamentoadpi/  
├─ config/          # SecurityConfig, DataBaseConfig, SwaggerConfig, RabbitMQConfig  
├─ controller/      # REST por domínio (agendamento, medico, paciente, ...)  
├─ service/         # regra de negócio, @Transactional, logging  
├─ repository/      # interfaces JpaRepository  
├─ model/  
│   ├─ entity/      # entidades JPA (Lombok)  
│   ├─ dto/         # records de saída  
│   ├─ request/     # records de entrada (Salvar.../Atualizar...)  
│   ├─ mapper/      # interfaces MapStruct  
│   ├─ response/    # PageResponse, MensagemSucessoResponse, TokenResponse  
│   └─ rabbitmq/    # eventos publicados (AgendamentoCriado/AtualizadoEvent)  
├─ exceptions/     # exceções + GlobalExceptionHandler  
└─ enums/          # TipoUsuario, SituacaoCadastro  
src/main/resources/db/migration/ # V1.0__CreateTables.sql, V1.1__Inserts.sql
```

Segurança e perfis de acesso

A autenticação é por **JWT**. São públicos apenas **POST /v1/auth/login**, a documentação OpenAPI e a Swagger UI. Todo o resto exige token. **Não há autocadastro**: criar qualquer cadastro exige um usuário autenticado. O perfil vem do **tipoUsuario**, prefixado com **ROLE_** e é comparado por **hasRole(...)**. Um token de usuário excluído é recusado, pois a exclusão desativa também as credenciais.

Recurso	Criar / Editar	Excluir	Listar / Consultar
Médico, enfermeiro, recepcionista	ADMINISTRADOR	ADMINISTRADOR	Todos, exceto PACIENTE
Paciente	Criar: RECEPTIONISTA · Editar: todos exceto PACIENTE	ADMINISTRADOR	Todos, exceto PACIENTE
Histórico do paciente	MEDICO ou RECEPTIONISTA	ADMINISTRADOR	MEDICO, RECEPTIONISTA ou ENFERMEIRO
Agendamento (consulta)	Marcar: MEDICO, RECEPTIONISTA ou ENFERMEIRO · Remarcar: RECEPTIONISTA	RECEPTIONISTA	MEDICO, RECEPTIONISTA, ENFERMEIRO ou PACIENTE

O paciente vê só as próprias consultas. O **GET /v1/agendamento** é o único endpoint acessível ao perfil PACIENTE. Quando quem chama é um paciente, a resposta traz apenas as consultas dele. Os demais papéis veem todas. O administrador, por decisão de negócio, não acessa consultas nem histórico (exceto para excluir histórico).

Endpoints da API

Método	Rota	Autorização
POST	/v1/auth/login	Pública
GET	/v1/medico	Exceto PACIENTE
POST	/v1/medico	ADMINISTRADOR
PATCH	/v1/medico/{id}	ADMINISTRADOR
DELETE	/v1/medico/{id}	ADMINISTRADOR
GET	/v1/enfermeiro	Exceto PACIENTE
POST	/v1/enfermeiro	ADMINISTRADOR
PATCH	/v1/enfermeiro/{id}	ADMINISTRADOR
DELETE	/v1/enfermeiro/{id}	ADMINISTRADOR

Método	Rota	Autorização
GET	/v1/recepcionista	Exceto PACIENTE
POST	/v1/recepcionista	ADMINISTRADOR
PATCH	/v1/recepcionista/{id}	ADMINISTRADOR
DELETE	/v1/recepcionista/{id}	ADMINISTRADOR
GET	/v1/paciente	Exceto PACIENTE
POST	/v1/paciente	RECEPCIONISTA
PATCH	/v1/paciente/{id}	Exceto PACIENTE
DELETE	/v1/paciente/{id}	ADMINISTRADOR
GET	/v1/historico-paciente	MEDICO, RECEPCIONISTA, ENFERMEIRO
POST	/v1/historico-paciente	MEDICO ou RECEPCIONISTA
PATCH	/v1/historico-paciente/{id}	MEDICO ou RECEPCIONISTA
DELETE	/v1/historico-paciente/{id}	ADMINISTRADOR
GET	/v1/agendamento	MEDICO, RECEPCIONISTA, ENFERMEIRO, PACIENTE
POST	/v1/agendamento	MEDICO, RECEPCIONISTA, ENFERMEIRO
PATCH	/v1/agendamento/{id}	RECEPCIONISTA
DELETE	/v1/agendamento/{id}	RECEPCIONISTA

As escritas respondem **MensagemSucessoResponse** e as listagens **PageResponse<XDTO>**. A documentação interativa está em </AgendamentoAPI/swagger-ui/index.html>.

Regras de negócio

- **Atualização parcial (PATCH).** Nenhum campo é obrigatório na atualização: envie apenas o que deseja alterar. Campos ausentes, assim como strings vazias ou só com espaços, preservam o valor atual. As validações de *formato* (e-mail, tamanho de CPF/telefone) continuam valendo para os campos enviados preenchidos.
- **Modo estrito do Jackson.** Um campo desconhecido no JSON derruba a requisição com **400**.
- **Exclusão lógica.** Excluir médico, enfermeiro, recepcionista ou paciente troca a **situacaoCadastro** para **EXCLUIDO** e desativa as credenciais. A partir daí o login responde **403** e o token emitido antes é recusado

com **401**. Histórico e agendamento não têm situação de cadastro, então sua exclusão é física.

- **Agenda do médico.** Ao marcar ou remarcar, exige-se um **intervalo mínimo de 1 hora** entre as consultas do médico. Um horário dentro do intervalo responde **409 Horário Indisponível!**. Exatamente 1 hora antes ou depois é aceito. Na remarcação, a própria consulta é desconsiderada.
- **Agenda do paciente.** Não é possível marcar nem remarcar se o paciente já tiver outra consulta no **mesmo** horário, e nesse caso responde **409 Horário de Consulta Indisponível!**.
- **Paginação.** Numeração baseada em 1, com os parâmetros **page**, **size** e **sort**.
- **Datas.** **LocalDate** como **dd/MM/yyyy** e **LocalDateTime** como **dd/MM/yyyy - HH:mm:ss**.

Mensageria de eventos de agendamento

Ao **criar (POST)** ou **remarcar (PATCH)** uma consulta, o serviço publica um evento na exchange **agendamento.events** (tipo *direct*), que roteia para a fila **notificacao.email.agendamento**. Quem consome essa fila é a **NotificacaoAPI** (seção 5), e a AgendamentoAPI não depende dela: apenas publica.

Evento	Routing key	Type-Id
Consulta criada	agendamento.criado	agendamento.criado.v1
Consulta remarcada	agendamento.atualizado	agendamento.atualizado.v1

Configuração. Host, porta e credenciais do broker vêm das variáveis **RABBITMQ_HOST**, **RABBITMQ_PORT**, **RABBITMQ_USERNAME** e **RABBITMQ_PASSWORD**. A exchange, a fila e os bindings são declarados pela aplicação na primeira publicação, e as mensagens são duráveis, em JSON.

Publicação após o commit. O evento só sai depois que a transação da consulta é confirmada (**@TransactionalEventListener** na fase **AFTER_COMMIT**), então uma operação desfeita não gera aviso ao paciente. Uma falha ao publicar é registrada em log e não desfaz a consulta já gravada.

Exemplos de requisição e resposta

1) Autenticação

```
POST /AgendamentoAPI/v1/auth/login
Content-Type: application/json
```

```
{
  "login": "fernanda.lima",
  "senha": "Teste@123"
}
```

```
HTTP/1.1 200 OK
{
  "token": "eyJhbGciOiJIUzI1NiJ9...",
  "tipo": "Bearer"
}
```

Credenciais do *seed* para teste local (todas com a senha **Teste@123**): **admin** (administrador), **joao.silva** (médico), **fernanda.lima** (recepcionista), **pedro.almeida** (paciente). O token vai no header **Authorization: Bearer <token>**.

2) Agendar uma consulta

```
POST /AgendamentoAPI/v1/agendamento
Authorization: Bearer <token>
Content-Type: application/json

{
  "medicoId": 1,
  "pacienteId": 1,
  "dataHoraConsulta": "25/08/2027 - 14:00:00",
  "observacao": "Primeira consulta cardiológica."
}
```

```
HTTP/1.1 201 Created
{
  "status": 201,
  "timestamp": "12/09/2026 - 15:32:58",
  "mensagem": "Consulta criada com sucesso!"
}
```

3) Evento publicado no RabbitMQ

```
// routing key: agendamento.criado - header __TypeId__: agendamento.criado.v1
{
  "eventId": "a9ab74d4-6cd2-4f20-8b4d-5a7aaee589ea",
  "agendamentoId": 10,
  "pacienteId": 1,
  "pacienteNome": "PEDRO ALMEIDA",
  "pacienteEmail": "pedro.almeida@email.com",
  "medicoId": 1,
  "medicoNome": "JOAO SILVA",
  "especialidade": "CARDIOLOGIA",
  "dataHoraConsulta": "2027-08-25T14:00:00",
  "observacao": "Primeira consulta cardiológica.",
  "ocorridoEm": "2026-09-12T15:32:58"
}
```

Erros comuns

Código	Quando ocorre
400	Corpo inválido, campo desconhecido ou formato incorreto (validação).
401	Token ausente, inválido, expirado ou de usuário excluído.
403	Perfil sem permissão para a operação, ou usuário inativo no login.
409	Conflito de agenda (médico ou paciente indisponível no horário).

Todo erro sai no formato `ErrorResponseDTO` com `status`, `title`, `instance`, `type`, `detail` e `timestamp`, produzido pelo `GlobalExceptionHandler`.

4. HistoricoAPI: Serviço de Histórico

API GraphQL, somente leitura, que consolida em uma única consulta os dados cadastrais do paciente, o histórico clínico e as consultas agendadas. Todas as rotas são relativas ao context path `/HistoricoAPI`.

Estrutura de pastas

```
src/main/java/br/com/fiap/historicoapi/  
├─ config/           # DataBaseConfig (dev, prod), SwaggerConfig  
├─ controller/       # HistoricoController: resolver GraphQL  
├─ service/          # HistoricoService: orquestra os repositórios  
├─ repository/       # paciente, agendamento, historicopaciente  
├─ model/  
│   ├─ entity/       # entidades JPA (mapeiam o schema existente)  
│   └─ dto/          # records de saída  
├─ exceptions/       # exceções, DTOs de erro e handler global  
└─ util/             # FormatadorData  
  
src/main/resources/graphql/      # query.graphqls, schema.graphqls  
src/test/resources/db/migration/ # migrations do perfil test (banco autossuficiente)
```

Endpoints

Por ser GraphQL, há um **único endpoint de dados**: a operação vai no corpo da requisição.

Endpoint	Método	Descrição
<code>/HistoricoAPI/graphql</code>	POST	Endpoint único de dados. Recebe <code>query</code> , <code>operationName</code> e <code>variables</code> , e devolve o envelope <code>{ data, errors }</code> .

Endpoint	Método	Descrição
<code>/HistoricoAPI/graphql</code>	GET	Interface interativa GraphQL, para explorar o schema no navegador.
<code>/HistoricoAPI/swagger-ui/index.html</code>	GET	Documentação Swagger UI.
<code>/HistoricoAPI/v3/api-docs</code>	GET	Documento OpenAPI bruto, em JSON.

Operação GraphQL

Operação	Tipo	Argumento	Retorno
<code>getHistoricoPaciente</code>	Query	<code>pacienteId: ID!</code>	Paciente com dados cadastrais e as listas <code>historico</code> e <code>consultas</code> .

Erros previstos

Situação	Classificação	Mensagem
Id nulo, zero ou negativo	<code>BAD_REQUEST</code>	O ID do Paciente deve ser um número inteiro positivo!
Paciente inexistente	<code>NOT_FOUND</code>	Paciente não encontrado - ID: {id}
Falha inesperada	<code>INTERNAL_ERROR</code>	Mensagem genérica. O detalhe fica apenas no log.

Por que o status é sempre 200. Seguindo a especificação GraphQL, o transporte HTTP responde `200 OK` mesmo quando a execução falha. O problema é sinalizado no array `errors`, com a classificação em `extensions.classification`. Requisições que sequer chegam ao resolver (uma URL inexistente, por exemplo) são tratadas pelo pipeline Spring MVC e respondem com o `ErrorResponseDTO` e o status HTTP correspondente.

Exemplos de requisição e resposta

Requisição: consulta do histórico

```
POST /HistoricoAPI/graphql
Content-Type: application/json

{
  "query": "query($id: ID!) { getHistoricoPaciente(pacienteId: $id) {
    \" id nome sobrenome cpf email telefone dataNascimento situacaoCadastro\"
    \" historico { queixaPrincipal historicoDoenca medicamentos alergias }\"
    \" consultas { nomeMedico dataHoraConsulta observacao } } }\",
  \"variables\": { \"id\": 1 }
}
```

Resposta 200 OK, paciente encontrado

```
{
  "data": {
    "getHistoricoPaciente": {
      "id": "1",
      "nome": "PEDRO",
      "sobrenome": "ALMEIDA",
      "cpf": "12345678901",
      "email": "pedro.almeida@email.com",
      "situacaoCadastro": "ATIVO",
      "historico": [
        { "queixaPrincipal": "Dor no peito",
          "medicamentos": "Losartana 50mg",
          "alergias": "Nenhuma alergia conhecida." }
      ],
      "consultas": [
        { "nomeMedico": "JOAO",
          "dataHoraConsulta": "22/08/2026 - 08:00:00",
          "observacao": "Primeira consulta cardiológica." }
      ]
    }
  }
}
```

Resposta para paciente inexistente

```
// variables: { "id": 999 }
{
  "errors": [
    { "message": "Paciente não encontrado - ID: 999",
      "path": [ "getHistoricoPaciente" ],
      "extensions": { "classification": "NOT_FOUND" } }
  ],
  "data": { "getHistoricoPaciente": null }
}
```

Formato dos dados. Campos **ID!** trafegam como String pela especificação GraphQL. As datas já saem formatadas pelo utilitário **FormatadorData**, nos padrões **dd/MM/yyyy** e **dd/MM/yyyy - HH:mm:ss**. Os dados lidos existem porque foram criados pelas migrations e pelos endpoints da AgendamentoAPI.

5. NotificacaoAPI: Serviço de Notificação

Serviço orientado a eventos, **sem interface HTTP**: o **application.yaml** define **spring.main.web-application-type: none**, então não há Swagger, context path nem porta publicada. Toda a entrada acontece

pela fila do RabbitMQ e por um job agendado. O serviço consome os eventos publicados pela AgendamentoAPI, avisa o paciente por e-mail e envia um lembrete na véspera da consulta.

Estrutura de pastas

```
src/main/java/br/com/fiap/notificacaoapi/  
├─ config/           # DataBaseConfig (dev, prod), RabbitMQConfig  
├─ enums/            # StatusNotificacao, TipoNotificacao  
├─ exceptions/       # NotificacaoNaoEncontradaException  
├─ model/              
│   ├── entity/      # Notificacao (entidade JPA)  
│   ├── mapper/       # NotificacaoMapper (MapStruct)  
│   └─ rabbitmq/      # eventos consumidos (AgendamentoCriado/AtualizadoEvent)  
├─ repository/       # NotificacaoRepository  
└─ service/            
    ├── notificacao/  # NotificacaoService, LembreteConsultaJob, NotificacaoSender  
    │                  # e EmailNotificacaoSender  
    └─ rabbitmq/      # AgendamentoEventListener: consumidor da fila  
src/main/resources/db/migration/ # V1.0__CreateTables.sql, V1.1__AlterEventIdLength.sql
```

Fluxo de notificação

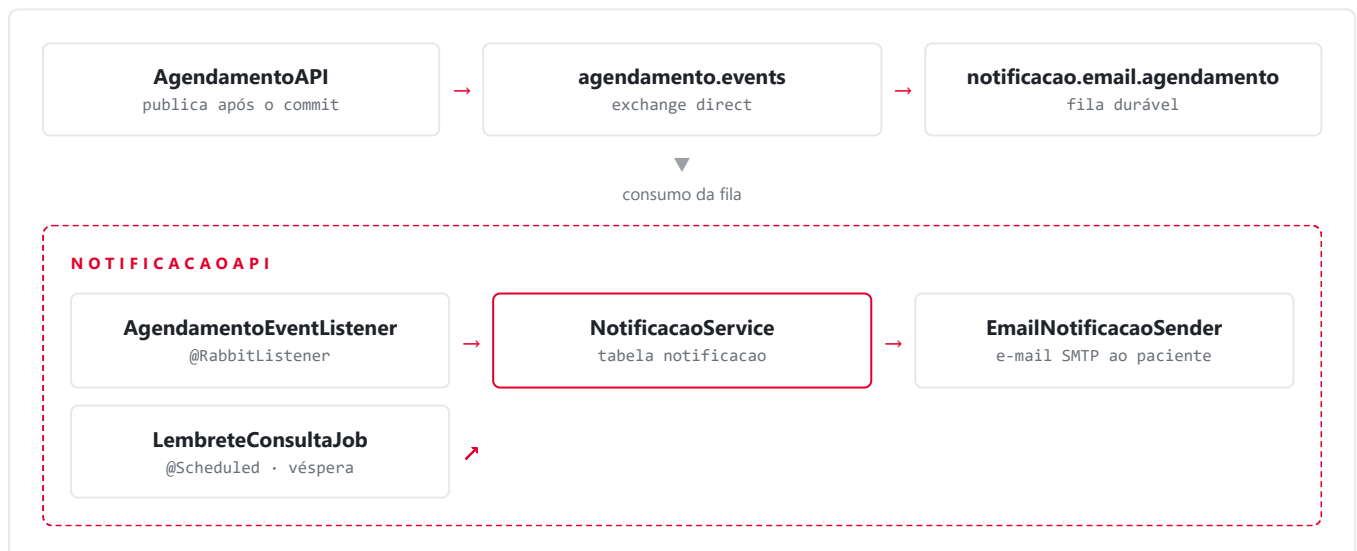


Figura 2. Do evento de agendamento ao e-mail do paciente

Toda notificação passa pelo mesmo ciclo, em três passos independentes:

- 1 **NotificacaoService.registrarPendente(...)** grava o registro com status **PENDENTE**, em uma transação própria.
- 2 **NotificacaoSender.enviar(...)** tenta enviar o e-mail, **fora** de qualquer transação de banco.
- 3 Conforme o resultado, **marcarComoEnviada(...)** grava **ENVIADA** e a data de envio, ou **marcarComoFalha(...)** grava **FALHA** e a mensagem de erro, cada um na sua própria transação.

Auditoria completa. Como o envio não participa da transação que criou o registro, uma falha no e-mail **nunca desfaz** a notificação gravada. A tabela **notificacao** guarda todo evento recebido junto com o seu desfecho. O envio fica atrás da interface **NotificacaoSender**, e **EmailNotificacaoSender** é hoje a única implementação.

Contrato de eventos

O **RabbitMQConfig** declara a exchange, a fila e os bindings **com os mesmos nomes e argumentos** da AgendamentoAPI, e registra um **DefaultClassMapper** com os mesmos Type IDs, apontando para as classes locais. Esse mapeamento é obrigatório: a AgendamentoAPI grava no header **__TypeId__** o Type ID, e não o nome completo da classe.

Routing key	Type ID	Classe	Notificação gerada
agendamento.criado	agendamento.criado.v1	AgendamentoCriadoEvent	CONSULTA_CRIADA
agendamento.atualizado	agendamento.atualizado.v1	AgendamentoAtualizadoEvent	CONSULTA_ATUALIZADA

O contrato é estrito. O Jackson está com **fail-on-unknown-properties: true**, então um campo novo no evento publicado pela AgendamentoAPI faz a desserialização falhar. Ao alterar os records de evento em um projeto, altere também no outro.

Tipos de notificação

Tipo	Origem	Assunto do e-mail
CONSULTA_CRIADA	Evento agendamento.criado	Consulta agendada com sucesso
CONSULTA_ATUALIZADA	Evento agendamento.atualizado	Sua consulta foi remarcada
LEMBRETE_CONSULTA	LembreteConsultaJob	Lembrete: sua consulta é amanhã

O remetente vem de **MAIL_FROM** (padrão **naoresponda@hospital.fiap.br**) e os assuntos ficam em **app.mail.***, no **application.yaml**. O e-mail de remarcação traz o horário anterior e o novo.

Resiliência e idempotência

- **Retry local.** **EmailNotificacaoSender.enviar(...)** tem **@Retryable** para **MailException**: 3 tentativas, com espera inicial de 2 segundos que dobra a cada nova tentativa, absorvendo falhas transitórias de SMTP sem devolver a mensagem à fila.
- **Persistência como fonte da verdade.** Se todas as tentativas falharem, o listener captura a exceção, grava **FALHA** com a mensagem de erro e confirma a mensagem normalmente. Isso evita um loop infinito de reentrega de uma mensagem que nunca seria processada.
- **Idempotência.** Antes de registrar, o serviço verifica se o **eventId** já existe e, se existir, ignora o evento sem reenviar o e-mail. A constraint **uk_notificacao_event_id** garante o mesmo no banco, inclusive numa

corrida entre duas entregas do mesmo evento.

- **Rejeição sem reenfileirar.** O container do listener usa `setDefaultRequeueRejected(false)`. Um erro que aconteça antes do registro, como falha de desserialização ou banco indisponível, descarta a mensagem em vez de devolvê-la à fila.

Não há Dead Letter Queue. O RabbitMQ exige que uma fila existente seja redeclarada com argumentos **idênticos**, ou recusa a conexão com `406 PRECONDITION_FAILED`. Como a fila também é declarada, sem DLQ, pela AgendamentoAPI, adicionar `x-dead-letter-exchange` só de um lado quebraria a integração. Para ter DLQ, os argumentos precisam mudar nos dois projetos ao mesmo tempo.

Lembrete de consultas

O `LembreteConsultaJob` busca as consultas marcadas para **o dia seguinte** e envia um e-mail de lembrete a cada paciente.

- A busca considera apenas o **último estado de cada agendamento**, o registro mais recente de `CONSULTA_CRIADA` ou `CONSULTA_ATUALIZADA`. Uma consulta remarcada para outro dia não gera lembrete pela data antiga.
- O lembrete é gravado com o `eventId lembrete-<eventId do último estado>`, então rodar o job de novo **não duplica** o envio.
- O envio segue o mesmo ciclo `PENDENTE` → `ENVIADA` ou `FALHA` das demais notificações.

Periodicidade. O job roda **diariamente às 8h** (`@Scheduled(cron = "0 0 8 * * *")`), no fuso `America/Sao_Paulo` definido pela variável `TZ` dos contêineres.

Cancelamentos. O serviço não lê as tabelas da AgendamentoAPI e o cancelamento de uma consulta não gera evento para a fila, então uma consulta cancelada ainda pode receber lembrete.

Banco de dados

O serviço é dono apenas da tabela `notificacao`, criada pelas próprias migrations no banco dedicado `notificacao`, dentro do mesmo servidor PostgreSQL. Cada registro guarda os dados necessários para montar o e-mail (paciente, médico, especialidade e horários), o que permite ao job de lembrete trabalhar sem consultar a AgendamentoAPI.

Migration	Descrição
<code>V1.0__CreateTables.sql</code>	Cria a tabela <code>notificacao</code> , com a constraint única <code>uk_notificacao_event_id</code> .
<code>V1.1__AlterEventIdLength.sql</code>	Amplia <code>event_id</code> para 60 caracteres, comportando o prefixo <code>lembrete-</code> somado ao UUID do evento.

Status	Significado
PENDENTE	Notificação registrada, envio ainda não concluído.
ENVIADA	E-mail enviado, com data_envio preenchida.
FALHA	Todas as tentativas falharam, com mensagem_erro preenchida.

Exemplo: e-mail de consulta agendada

Gerado a partir do evento publicado no exemplo 3 da seção 3.

```
// De: naoresponda@hospital.fiap.br · Para: pedro.almeida@email.com
// Assunto: Consulta agendada com sucesso
Olá, PEDRO ALMEIDA!

Sua consulta com JOAO SILVA (CARDIOLOGIA) foi agendada com sucesso.

Data e horário: 25/08/2027 às 14:00
Consulta: #10

Em caso de necessidade, entre em contato com a unidade responsável.

Atenciosamente,
Sistema Hospitalar
```

6. Configuração e Execução

Arquivos Docker Compose

Cada repositório traz Compose com papéis distintos. A separação é intencional: a infraestrutura não pertence a nenhum serviço em particular, então subir uma API não pode implicar subir um banco novo e isolado.

Arquivo	Papel
docker-compose-postgres-dev.yml	Infraestrutura de desenvolvimento, com credenciais fixas e sem .env . Cria a rede shared-net . Na AgendamentoAPI inclui PostgreSQL e RabbitMQ. Na HistoricoAPI, apenas o PostgreSQL. Na NotificacaoAPI, PostgreSQL, RabbitMQ e um contêiner de apoio que cria o banco notificacao .
docker-compose-postgres-prod.yml	Infraestrutura de produção: lê o .env e possui healthchecks. PostgreSQL nos três repositórios, e RabbitMQ na AgendamentoAPI e na NotificacaoAPI.
docker-compose-agendamentoapi.yml	Apenas a AgendamentoAPI, no perfil prod , publicando 9027:9027 . Consome shared-net como rede externa.

Arquivo	Papel
<code>docker-compose-historicoapi.yml</code>	Apenas a HistoricoAPI, no perfil prod , publicando 9028:9027 . Consome shared-net como rede externa.
<code>docker-compose-notificacaoapi.yml</code>	A NotificacaoAPI no perfil prod , sem porta publicada. Antes dela, o contêiner NotificacaoAPI-Database cria o banco notificacao caso ainda não exista. Consome shared-net como rede externa.

A definição do PostgreSQL é idêntica entre os três repositórios, com o mesmo nome de projeto Compose e de volume, então o container e os dados são sempre os mesmos, independentemente de qual repositório inicia a infraestrutura. Suba-a **uma única vez**. O RabbitMQ aparece, com definição idêntica, nos Compose da AgendamentoAPI, que publica, e da NotificacaoAPI, que consome. A HistoricoAPI não usa mensageria.

Configuração do ambiente (.env)

Em produção, as credenciais vêm de um arquivo **.env** na raiz de cada projeto. As variáveis criam a infraestrutura e conectam o serviço, de modo que não têm como divergir.

```
# AgendamentoAPI
DATABASE_NAME=postgres
DATABASE_USER=postgres
DATABASE_PASSWORD=*****
RABBITMQ_USERNAME=fiap
RABBITMQ_PASSWORD=*****
JWT_SECRET=uma-chave-longa-e-secreta
JWT_EXPIRATION_MS=3600000

# HistoricoAPI
DATABASE_NAME=postgres
DATABASE_USER=postgres
DATABASE_PASSWORD=*****

# NotificacaoAPI
DATABASE_USER=postgres
DATABASE_PASSWORD=*****
NOTIFICACAO_DATABASE_NAME=notificacao
RABBITMQ_USERNAME=fiap
RABBITMQ_PASSWORD=*****
MAIL_HOST=smtp.seuprovedor.com
MAIL_PORT=587
MAIL_USERNAME=usuario-smtp
MAIL_PASSWORD=*****
MAIL_FROM=naoresponda@hospital.fiap.br
```

Segredos. As senhas, o **JWT_SECRET** e as credenciais SMTP vivem apenas no **.env** (fora do controle de versão), nunca no código. Em produção, defina sempre um **JWT_SECRET** próprio. Na NotificacaoAPI, o Compose interrompe a subida com mensagem explícita se **MAIL_HOST**, **MAIL_USERNAME** ou **MAIL_PASSWORD** estiverem vazios.

Instruções para execução local

Pré-requisitos: **Java 21** (a toolchain do Gradle resolve a versão), **Docker** e **Docker Compose**, e os três repositórios clonados lado a lado. A AgendamentoAPI executa as migrations e cria o schema, então deve rodar antes da HistoricoAPI contra um banco novo. A NotificacaoAPI precisa ainda de um servidor SMTP, e em desenvolvimento um *sandbox* de e-mail evita envios reais aos pacientes.

Desenvolvimento (aplicações pela IDE, perfil dev)

```
# 1. Infraestrutura de desenvolvimento (uma vez). Pela NotificacaoAPI, o Compose
#   sobe PostgreSQL e RabbitMQ e também cria o banco notificacao
cd NotificacaoAPI
docker compose -f docker-compose-postgres-dev.yml up -d --wait

# 2. AgendamentoAPI: aplica as migrations e cria o schema + carga inicial (porta 9017)
cd ../AgendamentoAPI && ./gradlew bootRun --args="--spring.profiles.active=dev"

# 3. HistoricoAPI: le o mesmo banco (porta 9017)
cd ../HistoricoAPI && ./gradlew bootRun --args="--spring.profiles.active=dev"

# 4. NotificacaoAPI: banco notificacao, broker e SMTP por variáveis de ambiente
cd ../NotificacaoAPI
export DATABASE_IP=localhost DATABASE_PORT=8745 DATABASE_NAME=notificacao
export DATABASE_USER=postgres DATABASE_PASSWORD=*****
export RABBITMQ_USERNAME=fiap RABBITMQ_PASSWORD=*****
export MAIL_HOST=... MAIL_USERNAME=... MAIL_PASSWORD=...
./gradlew bootRun --args="--spring.profiles.active=dev"
```

Produção (tudo em contêiner, preencher antes o .env)

```
# 1. Infraestrutura: cria a rede shared-net (a partir da AgendamentoAPI, que traz o broker)
cd AgendamentoAPI
docker compose -f docker-compose-postgres-prod.yml up -d --wait

# 2. AgendamentoAPI (aplica migrations, publica em 9027)
docker compose -f docker-compose-agendamentoapi.yml up -d

# 3. HistoricoAPI (publica em 9028)
cd ../HistoricoAPI
docker compose -f docker-compose-historicoapi.yml up -d

# 4. NotificacaoAPI (cria o banco notificacao, aplica migrations e consome a fila)
cd ../NotificacaoAPI
docker compose -f docker-compose-notificacaoapi.yml up -d
```

```
# Testes de integração (basta a infraestrutura no ar)
./gradlew test
```

Windows. Utilize `.\gradlew.bat` no lugar de `./gradlew`. Na AgendamentoAPI e na HistoricoAPI, o IntelliJ traz as configurações prontas em `.run/` (BootRun DEV, BootRun PROD, Clean Build e Testes de Integração), que já injetam as variáveis de ambiente.

Testes da NotificacaoAPI. Os padrões do perfil `test` apontam para o PostgreSQL da fase (`localhost:8745`, banco `postgres`), os mesmos do CI. Contra o banco compartilhado, exporte `DATABASE_NAME=notificacao` antes de rodar a suíte, para que o Flyway não esbarre nas migrations da AgendamentoAPI.

URLs e portas configuradas

Recurso	URL	Porta
AgendamentoAPI (dev / prod)	<code>http://localhost:9017/AgendamentoAPI</code> · <code>:9027</code>	<code>9017 / 9027</code>
Swagger UI (Agendamento)	<code>/AgendamentoAPI/swagger-ui/index.html</code>	<code>9017 / 9027</code>
HistoricoAPI (dev / prod)	<code>http://localhost:9017/HistoricoAPI</code> · <code>:9028</code>	<code>9017 / 9028</code>
GraphiQL (Histórico)	<code>/HistoricoAPI/graphiql</code>	<code>9017 / 9028</code>
NotificacaoAPI	Sem HTTP. Acompanhamento pelos logs e pelo painel do RabbitMQ	—
PostgreSQL	<code>jdbc:postgresql://localhost:8745/postgres</code>	<code>8745 / 5432</code>
Banco da NotificacaoAPI	<code>jdbc:postgresql://localhost:8745/notificacao</code>	<code>8745 / 5432</code>
RabbitMQ (AMQP / painel)	<code>amqp://localhost:5672</code> · <code>http://localhost:15672</code>	<code>5672 / 15672</code>

Em cada par de portas, a primeira é a publicada no host e a segunda é a interna do contêiner ou da rede. As APIs HTTP sempre escutam em `9027` dentro do contêiner.

Perfis da aplicação

Perfil	Características
<code>dev</code>	SQL formatado, Hibernate em DEBUG. Log apenas no console. Porta 9017 nas APIs HTTP.
<code>prod</code>	Log em arquivo rotativo diário em <code>/app/logs</code> , com expurgo após 30 dias. Porta 9027 no contêiner nas APIs HTTP.
<code>test</code>	Perfil das suítes de integração, roda contra banco real. Na HistoricoAPI, o único em que o Flyway está habilitado. Na NotificacaoAPI, traz também padrões para o RabbitMQ e o SMTP.

7. Qualidade do Código

Os três projetos adotam as convenções idiomáticas do Spring Boot e princípios consolidados de projeto orientado a objetos. As suítes de testes servem de rede de segurança e sobem o contexto do Spring contra um banco real.

■ AgendamentoAPI

157 métodos de teste em 24 classes, com **100% de cobertura** de instruções e de branches em controllers, services e repositories.

JaCoCo · JUnit 5

■ HistoricoAPI

32 métodos de teste em 10 classes, com **100% de cobertura** de instruções, linhas, métodos, classes e branches.

JaCoCo · JUnit 5

■ NotificacaoAPI

13 métodos de teste em 3 classes. O build verifica a cobertura e **falha abaixo de 80%**.

JaCoCo · JUnit 5 · Mockito

Princípios SOLID e DRY

- **Responsabilidade única.** Cada camada tem um único motivo para mudar: o controller expõe a operação, o serviço concentra a regra de negócio, os repositórios cuidam do acesso a dados e os DTOs definem o contrato de saída. Na NotificacaoAPI, o listener só recebe, o serviço decide e o sender só envia.
- **Inversão de dependência.** Os serviços dependem de interfaces (**JpaRepository** e, na NotificacaoAPI, **NotificacaoSender**), nunca de implementações concretas, e quem as fornece é o container do Spring, por injeção via construtor com **@RequiredArgsConstructor** e campos **final**.
- **Aberto/fechado.** Novos domínios ou consultas entram como novas fatias verticais ou novos resolvers, sem alterar o que já existe. Uma nova exceção de negócio entra como um novo handler. Um novo canal de notificação, como SMS ou push, entra como nova implementação de **NotificacaoSender**, sem tocar no serviço nem no listener.
- **DRY.** O **GlobalExceptionHandler** concentra o tratamento de erros, os DTOs expõem uma *factory* única de conversão, a formatação de datas é centralizada e as classes-base de teste concentram as anotações comuns.

Modelagem, segurança e integração contínua

- **Imutabilidade e camadas isoladas.** DTOs, requests e eventos são **record** imutáveis, e a entidade JPA nunca é serializada diretamente, isolando o contrato público do mapeamento de banco.
- **Carregamento explícito.** Associações são **LAZY** e carregadas dentro da transação, evitando N+1 e **LazyInitializationException**.
- **Segurança da informação.** Senhas com BCrypt e nunca expostas, mensagens de exceções não-de-domínio não voltam ao cliente, e **include-stacktrace: never** impede vazamento de stacktrace. Nenhum segredo no código.

- **Testes de integração reais** sobem o contexto do Spring contra um PostgreSQL de verdade, não um banco em memória. Na `NotificacaoAPI`, o único mock desses testes é o `NotificacaoSender`, para não depender de um SMTP real.
- **Integração contínua.** Nos três projetos, a cada Pull Request para `main`, o GitHub Actions provisiona um PostgreSQL 18, configura o Java 21 e executa `./gradlew build`, e uma regressão reprova o PR antes do merge.

8. Repositórios do Código

URLs

- **AgendamentoAPI:** <https://github.com/JavaPosTech/AgendamentoAPI>
- **HistoricoAPI:** <https://github.com/JavaPosTech/HistoricoAPI>
- **NotificacaoAPI:** <https://github.com/JavaPosTech/NotificacaoAPI>
- Branch de entrega: `main` · Versão: `1.0.0`

Organização de cada repositório

Caminho	Conteúdo
<code>/src/main/java</code>	Código-fonte, organizado por camada e por domínio
<code>/src/main/resources</code>	<code>application.yaml</code> , <code>log4j2.xml</code> . Migrations (Agendamento e Notificação) ou schemas GraphQL (Histórico)
<code>/src/test</code>	Suítes de teste, e migrations do perfil de teste na HistoricoAPI
<code>docker-compose-postgres-*.yaml</code>	Infraestrutura de dev e de prod (PostgreSQL, e RabbitMQ na Agendamento e na Notificação)
<code>docker-compose-<api>.yaml</code>	Orquestração do próprio serviço no perfil prod
<code>Dockerfile</code>	Build multi-stage da imagem da aplicação
<code>.github/workflows</code>	Pipeline de integração contínua a cada Pull Request
<code>README.md</code> · <code>CLAUDE.md</code>	Documentação de execução e de arquitetura do projeto

Serviços da Fase 3

Serviço	Porta (host)	Papel
AgendamentoAPI	9027	Cadastros, autenticação, agendamento e histórico. Dona das migrations e publicadora dos eventos
HistoricoAPI	9028	Consulta consolidada do histórico do paciente, por GraphQL. Somente leitura
NotificacaoAPI	Sem porta	Consumidora dos eventos: e-mail ao paciente e lembrete da véspera. Dona da tabela notificacao

Infraestrutura compartilhada: PostgreSQL 18 (host **8745**) e RabbitMQ (host **5672**, painel **15672**), ambos na rede Docker **shared-net**. A AgendamentoAPI é o serviço dono do schema e deve subir primeiro contra um banco novo. A NotificacaoAPI usa o banco dedicado **notificacao** no mesmo servidor.